

EXPERIMENT – 1

1. Write a Java program that implements Quick sort algorithm for sorting a list of names in ascending order

```
class Main {

    static int partition(String[] names, int low, int high) {
        String pivot = names[high];
        int i = low - 1;

        for (int j = low; j <= high - 1; j++) {
            if (names[j].compareTo(pivot) < 0) {
                i++;
                swap(names, i, j);
            }
        }

        swap(names, i + 1, high);
        return i + 1;
    }

    static void swap(String[] names, int i, int j) {
        String temp = names[i];
        names[i] = names[j];
        names[j] = temp;
    }

    static void quickSort(String[] names, int low, int high) {
        if (low < high) {
            int pivot = partition(names, low, high);
            quickSort(names, low, pivot - 1);
            quickSort(names, pivot + 1, high);
        }
    }

    public static void main(String[] args) {
        String[] names = {
            "Anvesh", "Kaushik", "Mahira", "Shweta", "Prachi", "Shubham", "Gurvir", "Ibrahim"
        };

        System.out.println("Before Sorting: ");
        for (int i = 0; i < names.length; i++) {
            System.out.println(names[i]);
        }

        quickSort(names, 0, names.length - 1);

        System.out.println();
        System.out.println("After Sorting: ");
    }
}
```

```
        for (int i = 0; i < names.length; i++) {  
            System.out.println(names[i]);  
        }  
    }  
}
```

OUTPUT:

```
Before Sorting:  
Anvesh  
Kaushik  
Mahira  
Shweta  
Prachi  
Shubham  
Gurvir  
Ibrahim
```

```
After Sorting:  
Anvesh  
Gurvir  
Ibrahim  
Kaushik  
Mahira  
Prachi  
Shubham  
Shweta
```

Figure 1: Output of program 1

EXPERIMENT – 2

2. Write a Java program that implements Bubble sort algorithm for sorting in descending order and also shows the number of interchanges occurred for the given set of integers.

```
public class BubbleSort {
    static void sort(int[] a) {
        int interchange = 0;

        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] < a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                    interchange++;
                }
            }
        }

        System.out.println("Number of interchanges: " + interchange);
    }

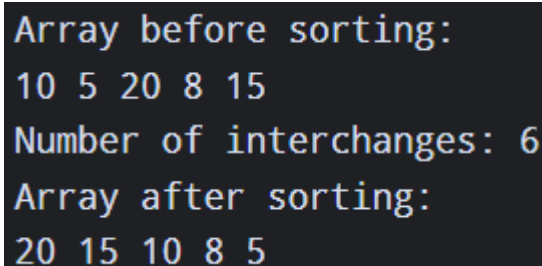
    public static void main(String[] args) {
        int[] a = {10, 5, 20, 8, 15};

        System.out.println("Array before sorting:");
        for (int i = 0; i < a.length; i++) {
            System.out.print(a[i] + " ");
        }
        System.out.println();

        sort(a);

        System.out.println("Array after sorting:");
        for (int i = 0; i < a.length; i++) {
            System.out.print(a[i] + " ");
        }
    }
}
```

OUTPUT:



```
Array before sorting:
10 5 20 8 15
Number of interchanges: 6
Array after sorting:
20 15 10 8 5
```

Figure 2: Output of program 2

EXPERIMENT – 3

3. Write a Java program that prompts the user for an integer and then prints out all the prime numbers up to that Integer.

```
import java.util.Scanner;

class PrimeNumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        System.out.println("Prime numbers from 1 to " + num + " are:");

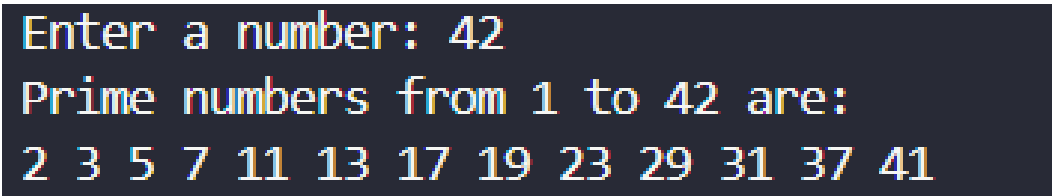
        for (int i = 2; i <= num; i++) {
            boolean isPrime = true;

            for (int j = 2; j * j <= i; j++) {
                if (i % j == 0) {
                    isPrime = false;
                    break;
                }
            }

            if (isPrime) {
                System.out.print(i + " ");
            }
        }

        sc.close();
    }
}
```

OUTPUT:

A screenshot of a terminal window with a dark background. It shows the output of a Java program. The first line is 'Enter a number: 42'. The second line is 'Prime numbers from 1 to 42 are:'. The third line is '2 3 5 7 11 13 17 19 23 29 31 37 41'.

```
Enter a number: 42
Prime numbers from 1 to 42 are:
2 3 5 7 11 13 17 19 23 29 31 37 41
```

Figure 3: Output of program 3

EXPERIMENT – 4

4. Write a Java program that checks whether a given string is a palindrome or not. Ex: MADAM is a palindrome.

```
import java.util.Scanner;

class Palindrome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

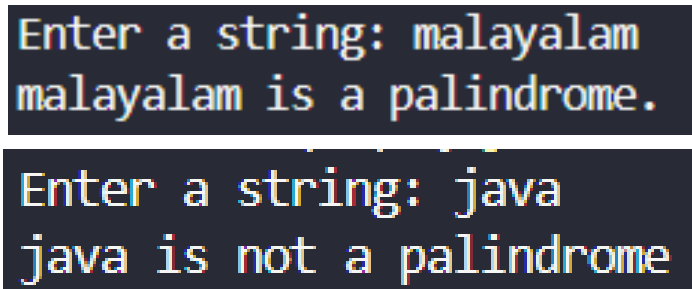
        System.out.print("Enter a string: ");
        String str = sc.next().toLowerCase();

        boolean isPalindrome = true;
        int j = str.length() - 1;
        for(int i = 0; i < str.length() / 2; i++) {
            if(str.charAt(i) != str.charAt(j--)) {
                isPalindrome = false;
                break;
            }
        }

        if(isPalindrome) {
            System.out.println(str + " is a palindrome.");
        } else {
            System.out.println(str + " is not a palindrome");
        }

        sc.close();
    }
}
```

OUTPUT:



```
Enter a string: malayalam
malayalam is a palindrome.

Enter a string: java
java is not a palindrome
```

Figure 4: Output of program 4

EXPERIMENT – 5

5. Write a Java program that works as a simple calculator. Use a grid layout to arrange buttons for the digits and for the +, -, *, % operations. Add a text field to display the result. Handle any possible exceptions like divided by zero.